

Katedra Elektrotechniki Teoretycznej i Informatyki		 Zachodniopomorski Uniwersytet Technologiczny w Szczecinie
Przedmiot: Struktury danych i techniki programowania		 Wydział Elektryczny
Numer ćwiczenia: 4	Temat: Programowanie obiektowe. Funkcje zaprzyjaźnione z klasami, przeciążanie operatorów.	

Deklaracje przyjaźni

Język C++ jest bardzo rozbudowany, ale dzięki temu elastyczny. Jak wspominaliśmy wcześniej, tworząc klasę dzielimy jej ciało na bloki opatrzone różnymi specyfikatorami dostępu: `private`, `public`, czy `protected`. Prywatne (jak również chronione) bloki klasy z definicji są niedostępne z innych poziomów programu. Niekiedy jednak taki dostęp jest nam z różnych względów potrzebny. Możemy wtedy uczynić część naszego kodu „przyjacielem” naszej klasy. *Przyjaciel (ang. friend) naszej klasy ma dostęp do wszystkich jej pól i metod, również tych opatrzonych specyfikatorem `private` lub `protected`.*

Składnia:

`friend deklaracja_przyjaciela;`

Mamy tutaj: słowo kluczowe `friend` oraz *deklaracja_przyjaciela*, czyli:

- Prototyp funkcji globalnej
- Prototyp funkcji zadeklarowanej w innej klasie
- Nazwa innej klasy

Uwaga: deklaracja przyjaźni nie podlega specyfikatorom dostępu. Nie ma więc znaczenia, w jakiej części klasy się ona znajdzie, zawsze będzie mieć takie samo znaczenie. Najczęściej deklaracje przyjaźni umieszcza się na początku lub na końcu klasy.

Funkcje zaprzyjaźnione

Chcąc uczynić jakąś funkcję przyjacielem klasy, musimy w definicji klasy podać deklarację zaprzyjaźnionej funkcji, poprzedzając ją słowem kluczowym `friend`.

Przykład 1 Przyjaźń funkcji z klasą

Klasa Point	
Point.cpp	Point.h
<pre>#include "Point.h" Point::Point() { } Point::Point(double X, double Y) { this->X = X; this->Y = Y; }</pre>	<pre>class Point { public: double X, Y; Point(); Point(double X, double Y); };</pre>

Klasa Circle	
Circle.cpp	Circle.h
<pre>#include "Circle.h" #include <cmath> bool IntersectCheck(Circle& C1, Circle& C2) // Definicja funkcji { double CentresDistance; CentresDistance = sqrt(pow(C1.Centre.X - C2.Centre.X, 2) + pow(C1.Centre.Y - C2.Centre.Y, 2)); return (CentresDistance < (C1.Radius + C2.Radius) && CentresDistance > abs(C1.Radius - C2.Radius)); } Circle::Circle(double X, double Y, double R):Centre(X,Y) { Radius = R; }</pre>	<pre>#include "Point.h" class Circle { private: Point Centre; double Radius; public: Circle(double X, double Y, double R); friend bool IntersectCheck(Circle&, Circle&); // funkcja IntersectCheck (sprawdza czy dwa okregi sie przecinaja) jest zaprzyjazniona z klasa Circle. Deklaracja };</pre>

- Funkcja zaprzyjaźniona nie jest metodą klasy! To znaczy, że nie ma wskaźnika this, obiekt klasy trzeba podać jej jako parametr.
- Deklaracja przyjaźni jest też deklaracją funkcji. To znaczy, że nie trzeba funkcji zaprzyjaźnionej dodatkowo deklarować w jakimś innym miejscu.
- Definicję funkcji zaprzyjaźnionej można wstawić od razu w klasie. Wygląda ona wtedy podobnie jak metoda składowa klasy, ale nią nie jest.

Pozostałe szczegóły na temat przyjaźni

Z klasą można zaprzyjaźnić również metodę innej klasy. Wtedy znowu metoda ta nie staje się automatycznie metodą klasy, z którą się przyjaźni. Składnia:

Przykład 2 Przyjaźń funkcji innej klasy z klasą

Klasa Point	
Point.cpp	Point.h
<pre>#include "Point.h" Point::Point() { } Point::Point(double X, double Y) { this->X = X; this->Y = Y; }</pre>	<pre>class Point { public: double X, Y; Point(); Point(double X, double Y); };</pre>
Klasa Circle	
Circle.cpp	Circle.h
<pre>#include "Circle.h" #include <cmath> Circle::Circle(double X, double Y, double R) :Centre(X, Y) { }</pre>	<pre>#include "Point.h" class Circle; class Control { public:</pre>

<pre> Radius = R; } bool Control::IntersectCheck(Circle& C1, Circle& C2) // Definicja funkcji { double CentresDistance; CentresDistance = sqrt(pow(C1.Centre.X - C2.Centre.X, 2) + pow(C1.Centre.Y - C2.Centre.Y, 2)); return (CentresDistance<(C1.Radius + C2.Radius) && CentresDistance>abs(C1.Radius - C2.Radius)); } </pre>	<pre> bool IntersectCheck(Circle&, Circle&); }; class Circle { private: Point Centre; double Radius; public: Circle(double X, double Y, double R); friend bool Control::IntersectCheck(Circle&, Circle&); // funkcja IntersectCheck (sprawdza czy dwa okregi się przecinaja) jest zaprzyjaźniona z klasa Circle. Deklaracja }; </pre>
---	---

Przyjaźń z klasą

Jeżeli chcemy, aby wszystkie metody pewnej klasy miały dostęp do wszystkich pól drugiej klasy, możemy zaprzyjaźnić klasy ze sobą.

Przykład 3 Przyjaźń klasy z klasą

Klasa Point	
Point.cpp	Point.h
<pre> #include "Point.h" Point::Point() { } Point::Point(double X, double Y) { this->X = X; this->Y = Y; } </pre>	<pre> class Point { public: double X, Y; Point(); Point(double X, double Y); }; </pre>
Klasa Circle	
Circle.cpp	Circle.h
<pre> #include "Circle.h" #include <cmath> Circle::Circle(double X, double Y, double R) :Centre(X, Y) { Radius = R; } bool Control::IntersectCheck(Circle& C1, Circle& C2) // Definicja funkcji { double CentresDistance; CentresDistance = sqrt(pow(C1.Centre.X - C2.Centre.X, 2) + pow(C1.Centre.Y - C2.Centre.Y, 2)); return (CentresDistance<(C1.Radius + C2.Radius) && CentresDistance>abs(C1.Radius - C2.Radius)); } </pre>	<pre> #pragma once #include "Point.h" class Circle; class Control { public: bool IntersectCheck(Circle&, Circle&); }; class LookIntoCircle { public: double GetRadius(Circle&); }; class Circle { private: Point Centre; } </pre>

<pre> } double LookIntoCircle::GetRadius(Circle& C) { return C.Radius; } </pre>	<pre> double Radius; public: Circle(double X, double Y, double R); friend class LookIntoCircle; friend bool Control::IntersectCheck(Circle&, Circle&); }; </pre>
---	--

Przeciążanie operatorów

Jednym z celów twórców języka C++ było zbliżenie funkcjonowania typów tworzonych przez programistę (czyli np. klasy, struktury, unie) i tych wbudowanych (takich jak int, czy double). Przeciążanie operatorów to najważniejszy z mechanizmów umożliwiających używanie swoich własnych klas podobnie jak typów wbudowanych, co oczywiście bardzo ułatwia programiście budowanie nawet bardzo skomplikowanego kodu.

Przeciążaniem operatorów nazywamy nadawanie istniejącym operatorom innego znaczenia. Przeciążanie jest wykonywane za pomocą tzw. **funkcji operatorowej**. Ogólna składnia funkcji operatorowej:

```

zwracany_typ operator symbol([parametry])
{
    ciało_funkcji
}

```

Słowo operator jest słowem kluczowym za którym należy wpisać symbol przeciążanego operatora. Jeżeli więc chcemy np. zdefiniować nowe znaczenie dla plusa (+), to piszemy funkcję operator+(). Funkcja operatorowa przyjmuje tyle argumentów, ile ma przeciążany przy jej pomocy operator. Do tych argumentów **zalicza się wskaźnik this**, jeżeli funkcja operatorowa jest metodą klasy.

Operatory możemy podzielić ze względu na ich cechy. W przypadku przeciążania operatorów liczba parametrów operatora jest szczególnie ważna. Operator można przeciążyć na kilka różnych sposobów:

- funkcja operatorowa może być funkcją składową klasy,
- funkcja może być funkcją globalną,
- funkcja może być zaprzyjaźniona z klasą.

Przeciążenie operatorów >> i <<

Przeciążenie operatora >> zwraca przez referencję obiekt typu istream& (cin jest obiektem typu istream). Ogólna składnia:

```
istream& operator>>(istream &in, NazwaKlasy &Obiekt) {}
```

Przeciążenie operatora << zwraca przez referencję obiekt typu ostream& (cout jest obiektem typu ostream). Ogólna składnia:

```
ostream& operator<<(ostream &out, NazwaKlasy &Obiekt) {}
```

Co istotne: przeciążenia operatorów `<<` i `>>` muszą być realizowane poprzez funkcje zewnętrzne (zaprzyjaźnione)

Czego nie możemy zmienić:

- Nie możemy tworzyć własnych operatorów
- Zmienić liczby argumentów, na których pracują operatory,
- Zmodyfikować priorytetu operatora, ani zmienić jego łączności

Przykład 4

Klasa Rational	
Rational.cpp	Rational.h
<pre>#include "Rational.h" Rational Rational::operator+(const Rational& U) const { Rational Wynik; Wynik.Licznik=U.Mianownik*Licznik+Mianownik*U.Licznik; Wynik.Mianownik=U.Mianownik*Mianownik; return Wynik; } std::ostream& operator<<(std::ostream& out, Rational& W) { out << W.Licznik << "/" << W.Mianownik; return out; } Rational operator-(const Rational& U1, const Rational& U2) { Rational Wynik; Wynik.Licznik=U2.Mianownik*U1.Licznik-U1.Mianownik*U2.Licznik; Wynik.Mianownik=U1.Mianownik*U2.Mianownik; return Wynik; }</pre>	<pre>#include <iostream> class Rational { private: int Licznik, Mianownik; public: Rational(int A = 1, int B = 1) { Licznik = A; Mianownik = B; } //Przeciążenie operatora + za pomocą funkcji składowej klasy Rational operator+(const Rational& U) const; Friend std::ostream& operator<<(std::ostream& out, Rational& W); Friend Rational operator- (const Rational&, const Rational&); };</pre>

Zadania:

1. Zaprzyjaźnij z klasą Circle funkcję sprawdzającą w której ćwiartce układu współrzędnych (albo na przecięciu których ćwiartek) znajduje się okrąg i drukującą o tym informację.

2. Zaprzyjaźnij z klasą Circle jeszcze dwie funkcje: CircleCompare i CircleAdd. Obie funkcje mają przyjmować jako argumenty dwa okręgi (obiekty typu Circle) i:
 - CircleCompare – porównywać ich promienie, w ten sposób, że zwracana jest wartość true, gdy promień obiektu pierwszego jest większy, niż promień obiektu drugiego
 - CircleAdd – Ma dodawać dwa okręgi do siebie, w tym sensie, że zwraca obiekt typu Circle o promieniu równym sumie promieni argumentów
 Utwórz kilka obiektów klasy i wykonaj różne operacje.
3. Zmodyfikuj klasę Circle tak, żeby funkcje CircleCompare i CircleAdd zostały zastąpione przeciążeniami odpowiednich operatorów (> i +)
4. Zmodyfikuj przykład 4 w ten sposób, że przeciążenie operatora << ma być realizowane tak, żeby zwracany był ułamek właściwy z wyłączonymi całościami (np. dla ułamka 10/3 wynik powinien być drukowany w postaci 3 1/3)

5.

```
class Complex
{
private:
    double Re, Im;
public:
    Complex(){}
    Complex(double InRe, double InIm):Re(InRe),Im(InIm)
    {}

    void Print()
    {
        cout<<Re<<"+"<<Im<<"i\n";
    }
};
```

Powyżej umieszczony został fragment kodu z zarysem klasy Complex, modelującej liczby zespolone. Uzupełnij klasę Complex:

- Przeciąż operatory arytmetyczne binarne: +, -, *, /, tak by możliwe były normalne operacje arytmetyczne na liczbach zespolonych (przeciążenie zrealizuj jako funkcje zaprzyjaźnione z klasą)
- Przeciąż operatory wyjścia i wejścia << i >>, tak by zastąpić funkcję Print().
- Przeciąż operator unarny -, w taki sposób, aby wynikiem było sprzężenie danej liczby zespolonej (przeciążenie zrealizuj jako funkcja składowa klasy)
- Zaprzyjaźnij z klasą funkcję Module, obliczającą moduł z danej liczby zespolonej ($|z| = \sqrt{z \cdot z^*}$), gdzie z^* - sprzężenie liczby z .